

VoxPath — Technical Documentation

A voice-native, multi-modal AI assistant for India. Talk or type to it and it answers from **live data** — Indian news (incl. Bihar & Uttar Pradesh regional), live stock/index prices, and Indian Railways (IRCTC) lookups — plus image generation and image-to-video. Built on **Google Vertex AI (Gemini)** with a single **MCP tool layer** shared by both the text and voice agents.

Table of contents

- 1. [What VoxPath is](#)
- 2. [High-level architecture](#)
- 3. [Technology stack](#)
- 4. [Repository layout](#)
- 5. [The MCP tool layer \(the heart of the app\)](#)
- 6. [Backend deep dive](#)
- 7. [Frontend deep dive](#)
- 8. [End-to-end request flows](#)
- 9. [HTTP & WebSocket API reference](#)
- 10. [Configuration & environment variables](#)
- 11. [Running the project locally](#)
- 12. [Data, storage & logging](#)
- 13. [External dependencies & accounts needed](#)
- 14. [Security notes](#)
- 15. [Glossary](#)

1. What VoxPath is

VoxPath is a **monorepo web application** that gives the user several ways to interact with an AI assistant, all backed by the same set of "live data" tools:

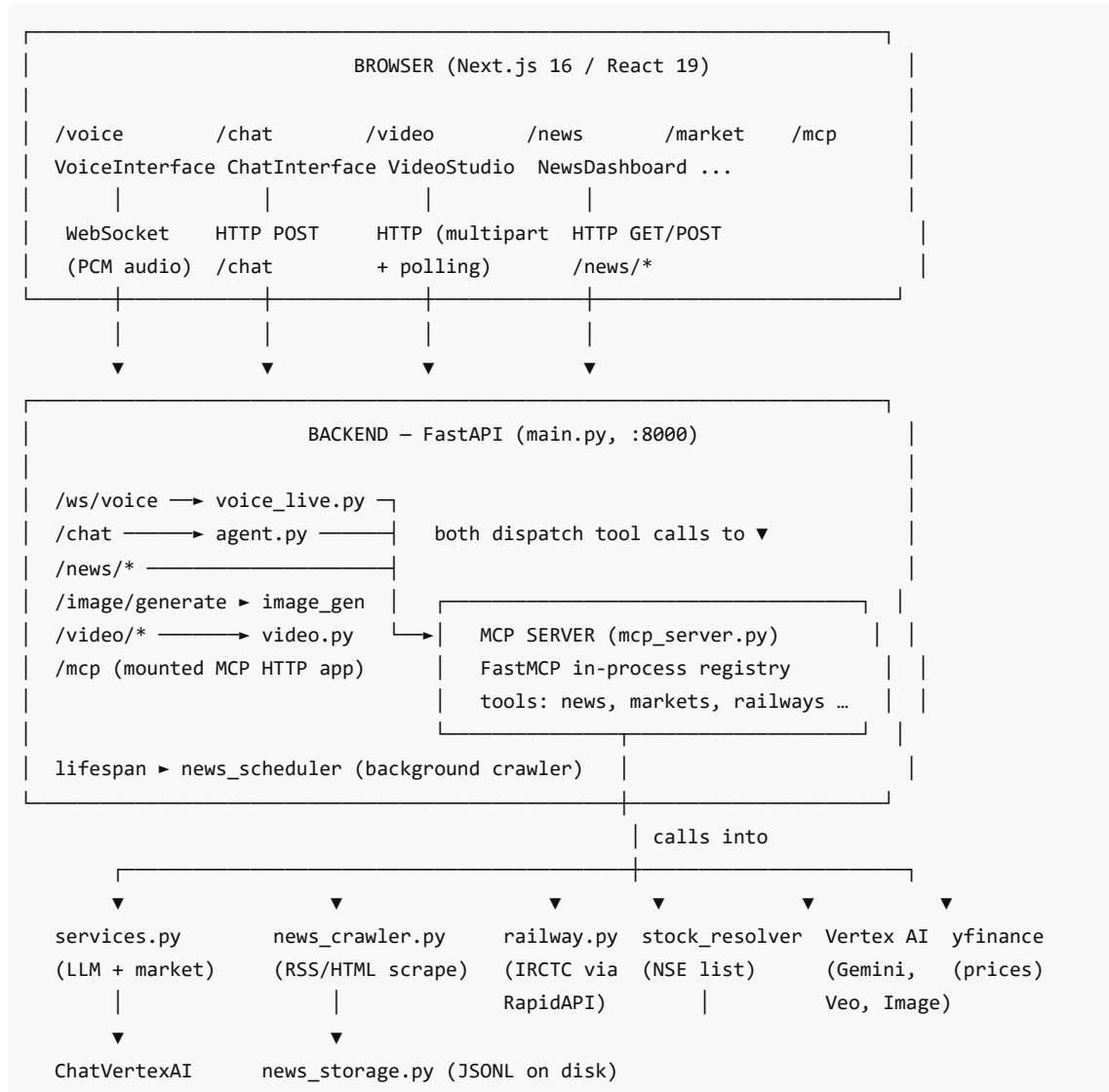
Surface	What it does
Voice	Real-time voice-to-voice conversation through Gemini Live (you speak, it speaks back). Speaks English & Hindi.
Chat	Text chat with a tool-using agent that routes your message to the right capability.
Image → Video	Generate an image from text (or upload one) and animate it into a short MP4 with Veo.
News Desk	Editorial dashboard of crawled Indian headlines with source/language/region filters, CSV export, and on-demand crawling.
Market Monitor	Indian stock/index price lookups and ticker extraction.
MCP Workspace	A view onto the underlying tool/resource registry.

The core design principle: **one tool registry, two agents**. Whether you talk (voice agent) or type (text agent), both call the *exact same* tools through an in-process **MCP server**. This guarantees consistent behaviour across surfaces

and a single place to add or change capabilities.

The assistant is explicitly told that **its training data is stale**, so for news, prices, and railway data it must **always call a tool** and answer only from the returned data — never from memory.

2. High-level architecture



Key components

- **Frontend (frontend/)** — Next.js App Router app. Each "workspace" is a route that renders one client component. Talks to the backend over HTTP and (for voice) a raw WebSocket.
- **API layer (backend/main.py)** — FastAPI app. Owns all HTTP/WebSocket endpoints, CORS, request tracing, and the background news scheduler lifecycle. It **mounts the MCP server** under `/mcp`.
- **MCP server (backend/mcp_server.py)** — A [FastMCP](#) server that declares every capability as an MCP **tool** (and a few **resources**). This is the single registry/executor used by both agents.
- **Agents** — `agent.py` (text, LangGraph ReAct) and `voice_live.py` (Gemini Live voice bridge). Both ultimately call MCP tools.

- **Capability modules** — `services.py` (LLM + market + news glue), `news_crawler.py` (+ `config/storage/scheduler`), `railway.py`, `stock_resolver.py`, `image_gen.py`, `video.py`.
- **Cross-cutting** — `logging_config.py` (structured + trace-id logging), `tool_format.py` (human-readable tool output rendering).

3. Technology stack

Backend (Python)

Area	Technology
Web framework	FastAPI (<code>fastapi[all]</code>) + Uvicorn ASGI server
Tool protocol	FastMCP (Model Context Protocol server + in-process client)
LLM / Agent	LangChain <code>langchain-google-vertexai</code> (ChatVertexAI) + LangGraph (<code>create_react_agent</code>)
Live voice & media	<code>google-genai</code> SDK → Gemini Live, Veo (video), Nano Banana (image), all on Vertex AI
Market data	<code>yfinance</code> (Yahoo Finance) + <code>pandas</code>
Railways	IRCTC API via RapidAPI (<code>irctc-api2.p.rapidapi.com</code>), <code>requests</code>
News scraping	<code>requests</code> , BeautifulSoup4 + lxml , <code>xml.etree</code> (RSS)
Config	python-dotenv (<code>.env</code>)
Auth to Google	Application Default Credentials (ADC) / service-account key

Frontend (TypeScript)

Area	Technology
Framework	Next.js 16 (App Router) + React 19
Language	TypeScript 5
Styling	Tailwind CSS v4 (+ <code>@tailwindcss/postcss</code>), <code>clsx</code> , <code>tailwind-merge</code>
Icons	lucide-react
Compiler	React Compiler (<code>babel-plugin-react-compiler</code>)
Browser audio	Web Audio API — <code>AudioContext</code> , <code>AudioWorklet</code> (PCM capture), <code>WebSocket</code>
Lint	ESLint 9 (<code>eslint-config-next</code>)

Tooling / orchestration

Area	Technology
Process orchestration	<code>concurrently</code> (npm) runs frontend + backend together
Launch scripts	PowerShell (<code>start_voxpath.ps1</code> , <code>run_voxpath.ps1</code>) for Windows
Platform	Developed on Windows 11 ; backend binds <code>127.0.0.1:8000</code> , frontend <code>:3000</code>

4. Repository layout

```
VoxPath/
├─ package.json          # root: "dev" script runs frontend+backend via concurrently
├─ start_voxpath.ps1     # installs deps then runs npm run dev
├─ run_voxpath.ps1       # checks npm/python on PATH then runs npm run dev
├─
├─ backend/
│   ├─ main.py           # FastAPI app: all HTTP/WS endpoints, CORS, tracing, lifespan
│   ├─ mcp_server.py      # FastMCP server – the single tool/resource registry
│   ├─ agent.py           # LangGraph ReAct text agent (Gemini via Vertex)
│   ├─ voice_live.py      # Gemini Live voice-to-voice WebSocket bridge
│   ├─ services.py        # LLM builder, routing, market + news glue, fallbacks
│   ├─ railway.py         # Indian Railways (IRCTC via RapidAPI) tool functions
│   ├─ stock_resolver.py  # NSE equity master list → resolve names/tickers
│   ├─ image_gen.py       # Text→image via "Nano Banana" (Gemini 2.5 Flash Image)
│   ├─ video.py           # Image→video via Veo (long-running, polled)
│   ├─ news_crawler.py    # RSS + HTML crawler, dedup, validation
│   ├─ news_config.py     # SOURCES list, user agents, thresholds
│   ├─ news_storage.py    # JSONL storage + seen-hash dedup persistence
│   ├─ news_scheduler.py  # Background periodic crawler (asyncio task)
│   ├─ logging_config.py  # Structured logging + per-request trace ids
│   ├─ tool_format.py     # Human-readable rendering of tool outputs for logs
│   ├─ requirements.txt   # Python dependencies
│   └─ data/news/        # Crawled headlines (headlines_YYYY-MM-DD.jsonl) +
seen_hashes.json
│   ├─ generated_videos/  # Veo output MP4s (job_id.mp4)
│   ├─ logs/              # voxpath.log, tool_calls.log, tool_calls_readable.log
│   ├─ check_vertex_live.py # Standalone Vertex Live connectivity check
│   ├─ nanobanana_demo.py # Standalone image-gen demo
│   ├─ crawl_up_news.py   # Helper / one-off crawl script
│   ├─ railway.py / test_*.py # tests: test_backend.py, test_units_fast.py
│   └─ ...
├─
├─ frontend/
│   ├─ package.json      # Next.js app deps + scripts (dev/build/start/lint)
│   ├─ next.config.ts, tsconfig.json, eslint.config.mjs, postcss.config.mjs
│   ├─ public/
│   │   └─ pcm-capture-worklet.js # AudioWorklet that captures 16kHz PCM mic frames
│   └─ src/
│       ├─ app/
│       │   ├─ layout.tsx      # Root layout: fonts, aurora background, FloatingNav
│       │   ├─ page.tsx        # Landing page (hero + SkillGrid)
│       │   ├─ globals.css     # Tailwind + design tokens / theme
│       │   ├─ voice/page.tsx  # → VoiceInterface
│       │   ├─ chat/page.tsx   # → ChatInterface
│       │   ├─ video/page.tsx  # → VideoStudio
│       │   ├─ news/page.tsx   # → NewsDashboard
│       │   ├─ market/page.tsx # market lookups
│       │   └─ mcp/page.tsx    # MCP workspace view
```

```

├── components/
│   ├── VoiceInterface.tsx    # Web Audio + WebSocket voice client
│   ├── ChatInterface.tsx     # Chat UI + /chat calls + health polling
│   ├── VideoStudio.tsx       # Image gen + video job start/poll + gallery
│   ├── NewsDashboard.tsx     # Headlines board, filters, crawl, CSV export
│   ├── SkillGrid.tsx         # Landing-page workspace cards
│   └── FloatingNav.tsx       # Top nav bar
└── store/                    # (reserved for client state)

nanobanana_share/            # Self-contained image-gen demo bundle (+ zip)
├── nanobanana_demo.py
├── requirements.txt
├── README.md
└── gcp_credentials.json     # 🚧 service-account key – see Security notes

```

5. The MCP tool layer (the heart of the app)

`backend/mcp_server.py` builds one `FastMCP` instance named **"VoxPath MCP"** and registers every capability on it. Both agents call tools through this single server via an **in-process MCP client** (`call_tool(name, args)`), so there is exactly one execution path and one place tools are defined.

Tools exposed

Tool	Purpose	Backed by
get_latest_news	Live Indian headlines; optional region/source topic	services → news_crawler
get_latest_headlines	Read stored headlines (filter by source/language)	news_crawler
crawl_news	Run the crawler now (all or matching sources)	news_crawler
list_news_sources	List configured sources	news_config
get_stock_price	Live price of any Indian-listed stock by name/ticker, or NIFTY	services + stock_resolver + yfinance
extract_market_tickers	LLM extraction of tickers from a query	services (Vertex)
market_report	Multi-ticker market summary	services
get_pnr_status	Train ticket PNR status (10-digit)	railway (IRCTC)
resolve_station_code	City/station name → IRCTC code (e.g. Delhi → NDLS)	railway
get_live_train_status	Live running status / delay / location	railway
get_train_schedule	Full route & timetable	railway
search_trains	All trains between two stations/cities	railway

check_seat_availability	Class-wise availability + confirmation chance	railway
get_fare	Class-wise fares for a train on a route	railway
route_chat, general_chat, handle_chat	Routing / general LLM chat helpers	services

Resources exposed

- `voxpath://health` — service health
- `voxpath://capabilities` — list of tools/resources
- `voxpath://news/sources` — configured news sources

Two agents, one registry

- **Text agent** (`agent.py`) loads a *subset* of MCP tools (`AGENT_TOOLS`) by pulling them from `mcp.get_tools()` and wrapping each as a LangChain `StructuredTool` , then builds a LangGraph ReAct agent.
- **Voice agent** (`voice_live.py`) declares the same capabilities as Gemini Live `FunctionDeclarations` (`VOICE_TOOLS`) and, when the model calls one, dispatches via `mcp_server.call_tool(...)` — the same executor.

The FastAPI app also **mounts the MCP server's own HTTP app** at `/mcp` (`app.mount("/mcp", mcp.http_app(path="/"))`), so the registry is reachable as a standard MCP endpoint too.

6. Backend deep dive

6.1 `main.py` — the FastAPI application

- **App + lifespan**: on startup it starts the background news scheduler; on shutdown it stops it (`lifespan` async context manager).
- **Request tracing middleware**: assigns a fresh 8-char trace id per request (via `logging_config.new_trace_id`) and logs method, path, status, and duration.
- **CORS**: origins from `ALLOWED_ORIGINS` (default `localhost:3000 / 127.0.0.1:3000`).
- **Endpoints** (full list in §9): `health`, `news` (`/news/*`), `chat` (`/chat`), `image` (`/image/generate`), `video` (`/video/*`), and the voice WebSocket (`/ws/voice`). It also mounts the MCP HTTP app at `/mcp` .
- **Quota handling**: `/chat` catches quota/rate-limit errors (`RESOURCE_EXHAUSTED / QUOTA / 429`) and returns a friendly fallback message instead of a 500.
- **Video jobs**: kept in an in-memory dict `video_jobs` keyed by a short job id; the actual MP4 is written to `generated_videos/<job_id>.mp4` .

6.2 `agent.py` — text agent (LangGraph ReAct)

- Single ReAct-style agent built with `create_react_agent(llm, tools, prompt)` .
- `llm` is `ChatVertexAI` (Gemini) from `services.build_llm()` .
- Tools are the `AGENT_TOOLS` subset pulled from the MCP registry.
- The **system prompt** makes it a concise Indian assistant that must call tools for news/markets/railways, resolve station codes before fare/availability calls, and ask for missing details (PNR, train number, stations, date).
- The agent is lazily built once and cached in a module global (`_agent`).

6.3 `voice_live.py` — Gemini Live voice bridge

This is the most involved module. It bridges one browser WebSocket to one Gemini Live session on Vertex AI.

- **Audio contract:** browser sends **16 kHz mono PCM16**; model returns **24 kHz mono PCM16**. Binary WS frames carry audio; JSON text frames carry control/transcript events.
- **Turn-taking:** relies on Gemini's **server-side voice activity detection** — the client just streams continuously. Barge-in ("interrupted") is surfaced to the browser so it can flush playback.
- **Two concurrent tasks:**
 - `browser_to_gemini` : forwards mic audio as `Blob` s; also handles `"end"` (audio-stream-end) and **typed input** (a JSON `{type:"text"}` frame — used when speech mis-hears a PNR/train number).
 - `gemini_to_browser` : streams model audio back; handles **tool calls** (dispatch → `execute_tool` → `mcp.call_tool` → send `FunctionResponse`), plus input/output **transcripts** and `turn_complete` .
- **Config** (`build_live_config`): `response_modalities=["AUDIO"]` , the system instruction, the `VOICE_TOOLS` function declarations, and input+output audio transcription enabled (so the UI can show what was said/heard).
- **Logging**: every tool call is logged three ways — a console line, a structured JSON line (`tool_calls.log`), and a human-readable block (`tool_calls_readable.log`).
- Defaults: model `gemini-live-2.5-flash` , location `global` (confirmed working on the lab project — see `check_vertex_live.py`).

6.4 `services.py` — LLM + market + news glue

- `build_llm()` → `ChatVertexAI` (model `gemini-2.5-flash` by default, `temperature=0` , `max_retries=0`), authenticated via ADC.
- **Routing** (`route_message`) is keyword-based: news keywords → news agent, market keywords (or "share") → market agent, else general.
- **Stock pricing** is robust:
 - `_candidate_symbols()` builds an ordered list of Yahoo symbols using (1) the official **NSE equity list** (`stock_resolver`), (2) **Yahoo Search** (handles renames/demergers, BSE-only names), and (3) naive `.NS` / `.BO` fallbacks. NSE is preferred over BSE; `NIFTY` maps to `^NSEI` .
 - `_price_for_symbol()` tries `fast_info.last_price` , then falls back to 5-day history close.
- **News helpers:** `get_latest_news` (curated, with general + crawl fallbacks), `news_chat` , language/region/limit detection from free text (incl. Hindi aliases like `सूची`).
- `build_fallback_text()` is the friendly message used when quota is exhausted.

6.5 `railway.py` — Indian Railways (IRCTC via RapidAPI)

- All calls go to `irctc-api2.p.rapidapi.com` and require `RAPIDAPI_KEY` .
- Functions: `get_pnr_status` , `resolve_station_code` , `get_live_train_status` , `get_train_schedule` , `search_trains` , `check_seat_availability` , `get_fare` .
- Rich **input validation** (PNR = 10 digits, train number = 4–5 digits) and **date normalization** (DD-MM-YYYY / DD-MM-YYYY ↔ API formats).
- A built-in **city→station-code map** (`CITY_STATION_MAP`) expands cities like "Delhi" → `[NDLS, ANVT, DLI, ...]` so searches cover all relevant terminals, with a `stationSearch` API fallback.
- Every function returns a plain dict (data or `{"error": ...}`), never raises to the caller.

6.6 `stock_resolver.py` — NSE symbol resolution

- Downloads NSE's official `EQUITY_L.csv` (~2,300 companies), cached in memory for 24h with a thread lock; keeps a stale cache if a refresh fails.

- `nse_matches()` scores candidates: exact ticker > exact name > name-prefix > substring > word-subset (ignoring noise words like "LIMITED", "LTD", "INDIA").

6.7 News subsystem

- `news_config.py` — the `SOURCES` list: ~17 sources mixing **RSS** (Times of India, The Hindu, NDTV, Hindustan Times, Jagran Hindi) and **HTML scrape** (BBC India, Amar Ujala, Live Hindustan, Dainik Jagran, News18 Hindi, Firstpost, plus dedicated **Bihar** and **Uttar Pradesh** city/region pages). Also tunables: `SIMILARITY_THRESHOLD=0.8` , `MAX_WORKERS=5` , `TIMEOUT=15` , rotating `USER_AGENTS` .
- `news_crawler.py` — the engine:
 - Per-domain **rate limiting** (1–2s jitter) with per-domain locks + retry adapter (3 retries on 5xx).
 - **RSS** parsed via `xml.etree` ; **HTML** via BeautifulSoup with CSS selectors (falls back to `h1/h2/h3` if selectors miss).
 - **Validation** (`is_valid_headline`) filters noise ("read more", "subscribe", too-short, self-links, etc.).
 - **Mojibake repair** + Unicode `NFKC` normalization for clean Hindi/English.
 - **Dedup**: SHA-256 hash of the headline (cross-run, persisted) plus a Jaccard-token **similarity** check (same language, > 0.8) within a batch.
 - Concurrency via a `ThreadPoolExecutor` .
- `news_storage.py` — append-only **JSONL per day** (`data/news/headlines_YYYY-MM-DD.jsonl`) + `seen_hashes.json` (atomic write).
- `news_scheduler.py` — an asyncio background loop that crawls every `NEWS_REFRESH_INTERVAL` seconds (default 300) so tools read pre-warmed headlines instantly. Exposes `status()` for the `/news/status` badge.

6.8 Media generation

- `image_gen.py` — text→image via "Nano Banana" (`gemini-2.5-flash-image`) on the Vertex `global` endpoint; returns PNG/JPEG bytes.
- `video.py` — image→video via **Veo** (`veo-3.0-generate-001` , `us-central1`). Veo is a **long-running operation**: start → poll every 10s (timeout 600s) → return MP4 bytes. Default motion prompt provided if none given.

6.9 Logging (`logging_config.py`)

- Console + rotating file (`logs/voxpath.log` , 5 MB × 5).
- **Per-request/voice-session trace id** carried in a `ContextVar` and printed in every log line (greppable by request).
- Two dedicated tool logs: structured JSON (`tool_calls.log`) for replay/audit and curated human-readable (`tool_calls_readable.log`).
- Level via `LOG_LEVEL` (default INFO; DEBUG adds audio counts, LLM prompts, etc.).

7. Frontend deep dive

A Next.js App Router app. `layout.tsx` sets up Google fonts (Geist, Inter, DM Serif Display), an animated "aurora" background, and the `FloatingNav` bar, then renders the active route. The landing page (`page.tsx`) shows a hero + the `SkillGrid` (one card per workspace).

All components read the backend base URL from `process.env.NEXT_PUBLIC_API_BASE` (default `http://127.0.0.1:8000`).

7.1 `VoiceInterface.tsx` (`/voice`)

The most technically dense component — a full real-time audio client:

- **Capture:** `getUserMedia` (with echo cancellation/noise suppression) → `AudioContext` @16 kHz → an **AudioWorklet** (`/pcm-capture-worklet.js`) that emits PCM16 frames → sent as binary WS frames.
- **Playback:** incoming binary frames are `Int16→Float32` converted and scheduled on a 24 kHz output `AudioContext` with a small (~60 ms) **jitter buffer** so back-to-back chunks don't glitch.
- **Half-duplex gate:** the mic is muted while the assistant is speaking (`sendingEnabledRef`) so it can't hear and interrupt itself; re-enabled after playback drains or on `turn_complete` . Barge-in (`interrupted`) flushes playback.
- **Control frames:** handles `ready` , `input_transcript` , `output_transcript` , `turn_complete` , `interrupted` , `error` . Shows live "You"/"VoxPath" transcripts.
- **Typed fallback:** a text box lets you type a PNR/train number if speech mis-hears it (sent as a JSON text frame).
- **News badge:** polls `/news/status` every 30s to show "Live news updated N min ago".

7.2 ChatInterface.tsx (/chat)

- Simple message list; POSTs `{message}` to `/chat` and appends the reply.
- Polls `/health` every 5s for an Online/Offline indicator.
- Shows a "thinking" state and surfaces backend errors inline.

7.3 VideoStudio.tsx (/video)

- Two modes: **Upload** an image, or **Generate** one from a prompt (`POST /image/generate` → blob → used as the source image).
- Starts a video job (`POST /video/generate` , multipart: image + motion prompt + aspect ratio), then **polls** `/video/status/{job_id}` every 5s, shows an elapsed timer, and on completion plays/downloads `/video/result/{job_id}` .
- Loads a **gallery** of previous generations from `/video/gallery` .
- Aspect ratios: 16:9 (Landscape), 9:16 (Reel), 1:1 (Square).

7.4 NewsDashboard.tsx (/news)

- Loads sources (`/news/sources`) and headlines (`/news/headlines?...`).
- Filters by **source**, **language** (All/English/Hindi), and **region segment** (All / Bihar / UP) — the Bihar/UP segments are computed **client-side** with curated keyword lists (incl. Devanagari) and exclusion logic to reduce false positives.
- "Run Fresh Crawl" calls `POST /news/crawl` ; "Refresh View" re-reads stored headlines. Includes a lead-story layout, top-headlines list, category/source breakdowns, and **CSV export** (with UTF-8 BOM for Excel).

7.5 Shared UI

- `SkillGrid.tsx` — the six workspace cards on the landing page.
- `FloatingNav.tsx` — top nav (hidden on `/`).
- Styling is Tailwind v4 with custom design tokens in `globals.css` (gradients, "surface-card", "orb-core", aurora blobs).

8. End-to-end request flows

8.1 Text chat

```
User types → ChatInterface POST /chat {message}
→ main.chat_endpoint → agent.agent_chat(message)
```

→ LangGraph ReAct agent (ChatVertexAI) decides to call a tool
→ StructuredTool wraps an MCP tool fn → services/railway/... → live data
→ agent composes final text answer
→ JSON {text} → rendered in chat
(quota error → friendly fallback text instead of 500)

8.2 Live voice

User speaks → mic 16kHz PCM (AudioWorklet) → WS binary → /ws/voice
→ voice_live forwards audio to Gemini Live (Vertex, global)
→ model VAD detects end of turn; may emit tool_call
→ execute_tool → mcp.call_tool → railway/services/... → result
→ FunctionResponse back to model
→ model streams 24kHz PCM answer → WS binary → browser schedules playback
→ transcripts + turn_complete sent as JSON control frames

8.3 Image → Video

(optional) prompt → POST /image/generate → Nano Banana → PNG blob (becomes source)
source image + motion prompt + ratio → POST /video/generate
→ background asyncio task → video.generate_video → Veo (start + poll) → MP4 on disk
client polls /video/status/{id} → on done → GET /video/result/{id} (MP4)

8.4 News

Startup: news_scheduler crawls every 5 min → JSONL on disk
User opens /news → GET /news/sources + /news/headlines (reads stored JSONL)
"Run Fresh Crawl" → POST /news/crawl → news_crawler (fetch, dedup, store)
Voice/Chat "latest news" → get_latest_news tool → reads stored headlines instantly

9. HTTP & WebSocket API reference

Base URL (dev): `http://127.0.0.1:8000`

Method	Path	Body / Params	Returns
GET	/health	—	{status:"ok"}
GET	/news/status	—	{last_refresh, age_seconds, interval_seconds}
GET	/news/sources	—	{sources:[{name,language,type}]}
GET	/news/headlines	limit, source?, language?	{status,count,headlines[]}
POST	/news/crawl	{source_query?, limit?, dry_run}	crawl summary {saved, total_extracted, duplicates, headlines[...]}
POST	/chat	{message}	{text, audio:null}

POST	/image/generate	{prompt}	PNG bytes (image/png)
POST	/video/generate	multipart: image, prompt, aspect_ratio	{job_id, status:"running"}
GET	/video/status/{job_id}	—	{job_id, status, ...}
GET	/video/result/{job_id}	—	MP4 file (video/mp4)
GET	/video/gallery	—	{videos:[{job_id,modified,bytes}]}
WS	/ws/voice	binary PCM16 in/out + JSON control	live voice session
(mount)	/mcp	MCP protocol	MCP HTTP app

Voice WebSocket frames

- Client → server: binary 16 kHz PCM16; text "end" ; JSON {type:"text", text} .
- Server → client: binary 24 kHz PCM16; JSON {type} where type ∈ ready | input_transcript | output_transcript | turn_complete | interrupted | error .

10. Configuration & environment variables

Backend reads a `backend/.env` (via `python-dotenv`). All Vertex calls use **Google Application Default Credentials** — set `GOOGLE_APPLICATION_CREDENTIALS` to a service-account JSON, or run `gcloud auth application-default login` .

Variable	Default	Used by
GOOGLE_CLOUD_PROJECT / GCP_PROJECT_ID	— (required)	all Vertex calls
GOOGLE_APPLICATION_CREDENTIALS	—	ADC auth (service-account key path)
VERTEX_MODEL	gemini-2.5-flash	text agent LLM
VERTEX_LOCATION	us-central1	text agent LLM
VERTEX_LIVE_MODEL	gemini-live-2.5-flash	voice
VERTEX_LIVE_LOCATION	global	voice (Live needs global)
IMAGE_MODEL	gemini-2.5-flash-image	image gen
IMAGE_LOCATION	global	image gen
VEO_MODEL	veo-3.0-generate-001	video
VEO_LOCATION	us-central1	video
RAPIDAPI_KEY	— (required for railways)	railway.py

ALLOWED_ORIGINS	http://localhost:3000,http://127.0.0.1:3000	CORS
NEWS_REFRESH_INTERVAL	300 (seconds)	background crawler
LOG_LEVEL	INFO	logging
TOOL_LOG_MAX	1500 (chars; 0 = unlimited)	voice tool-call log truncation
NEXT_PUBLIC_API_BASE (frontend)	http://127.0.0.1:8000	all frontend API calls

Memory note: the project's working Vertex config (model/region, and that Live requires the `global` endpoint) is recorded in the project memory under "Vertex AI config".

11. Running the project locally

Prerequisites

- **Node.js** (with npm) and **Python 3.11+** on `PATH` .
- A **Google Cloud project** with Vertex AI enabled + credentials (ADC).
- A **RapidAPI key** subscribed to the IRCTC API (for railway features).
- A `backend/.env` with at least `GOOGLE_CLOUD_PROJECT` and `RAPIDAPI_KEY` .

One-shot (Windows / PowerShell)

```
# Installs backend (pip) + frontend (npm) deps, then launches both:
./start_voxpath.ps1
```

Manual

```
# Backend
cd backend
python -m pip install -r requirements.txt
python main.py           # serves on http://127.0.0.1:8000

# Frontend (new terminal)
cd frontend
npm install
npm run dev              # serves on http://localhost:3000
```

Or via the root script (runs both with concurrently)

```
npm install           # root: installs concurrently
npm run dev           # runs frontend dev + backend together
```

Then open <http://localhost:3000>.

Tests

```
cd backend
python -m pytest test_units_fast.py # fast unit tests
python -m pytest test_backend.py   # backend tests
```

12. Data, storage & logging

What	Where
Crawled headlines	backend/data/news/headlines_YYYY-MM-DD.jsonl (append-only, one JSON per line)
Dedup hashes	backend/data/news/seen_hashes.json
Generated videos	backend/generated_videos/<job_id>.mp4
Generated image (demo)	backend/nanobanana_output.png
App log	backend/logs/voxpath.log (rotating, 5 MB × 5)
Tool calls (JSON)	backend/logs/tool_calls.log
Tool calls (readable)	backend/logs/tool_calls_readable.log

There is **no database** — state is files on disk plus in-memory dicts (`video_jobs` , the cached agent, the NSE list cache). Video job metadata is in-memory only (lost on restart), though the MP4 files persist and are listed by the gallery endpoint.

13. External dependencies & accounts needed

Service	Why	Auth
Google Vertex AI	Gemini (text & Live voice), Veo (video), Nano Banana (image)	ADC / service account
RapidAPI – IRCTC API (irctc-api2)	All Indian Railways data	RAPIDAPI_KEY header
Yahoo Finance (yfinance)	Live stock/index prices & symbol search	none
NSE archives	Official equity master list for symbol resolution	none
News sites (RSS/HTML)	Headline crawling	none (rate-limited, rotating UA)

14. Security notes

These are worth addressing before this is shared or deployed.

- **Committed credentials:** `nanobanana_share/gcp_credentials.json` is a Google Cloud **service-account key** sitting in the working tree (and inside `nanobanana_share.zip`). Treat it as **compromised** — rotate/revoke the key, remove the file, and add it (and all `.env` files) to `.gitignore` before the first commit. Never commit secrets.

- **CORS + credentials:** `allow_credentials=True` with a configurable origin list — keep `ALLOWED_ORIGINS` tight in production (don't use `*`).
- **No auth on the API:** every endpoint (including `/chat`, voice, media generation) is open. Anyone who can reach the backend can spend your Vertex/Veo and RapidAPI quota. Add authentication and rate limiting before exposing it.
- **Server-side scraping & API keys:** the RapidAPI key and all model access live server-side (good) — keep them out of the frontend bundle (only `NEXT_PUBLIC_*` vars are shipped to the browser).
- **In-memory job store:** `video_jobs` and the agent cache are per-process; they won't survive restarts or scale horizontally without shared state.

15. Glossary

Term	Meaning
MCP	Model Context Protocol — a standard way to expose tools/resources to LLMs. Here, <code>FastMCP</code> is the in-process registry both agents use.
Gemini Live	Google's low-latency, bidirectional voice (and multimodal) model API, used for the real-time voice surface.
Veo	Google's video-generation model (image→video here), run as a long-running Vertex operation.
Nano Banana	Nickname for Gemini 2.5 Flash Image, used for text→image.
ADC	Application Default Credentials — Google's standard auth resolution (env var key file or <code>gcloud</code> login).
ReAct agent	An LLM agent loop that interleaves reasoning and tool calls; built here with <code>LangGraph</code> .
IRCTC	Indian Railway Catering and Tourism Corporation — source of the railway data (via a RapidAPI wrapper).
NSE / BSE	National / Bombay Stock Exchange; symbols suffixed <code>.NS</code> / <code>.BO</code> on Yahoo Finance.
PCM16	16-bit signed little-endian raw audio samples — the wire format for voice.
Trace id	Short id attached to every log line within one request/voice session for easy correlation.

Generated as living documentation for the `VoxPath` codebase. Keep it updated as tools, models, and endpoints change.

...